

TUNABLE NON-LINEAR MANIFOLD ROMs FOR EL-LIPTIC, PARABOLIC, AND HYPERBOLIC PDES VIA MATRIX-FREE PETROV–GALERKIN PROJECTION

Anonymous authors

Paper under double-blind review

ABSTRACT

Neural operators such as Fourier Neural Operators and DeepONets deliver one (accuracy, speed) operating point per trained model; moving it requires retraining. We present a nonlinear-manifold reduced-order model (NM-ROM) that exposes a deployment-time accuracy/speed tradeoff from a single trained model. After training, its spatial bank is ordered once by importance, so a query chooses how many bank functions to use, and whether to solve with a nonlinear head or in the linear span of the bank, without changing a learned weight. The framework combines a coordinate-network bank with exact Dirichlet enforcement, least-squares Petrov–Galerkin projection of the discrete residual onto fixed tests with every linear term precomputed, matrix-free JAX evaluation, and empirical quadrature for the nonlinear term, confirmed on held-out states the solver reaches. On viscous Burgers at 4096^2 it reaches 0.22 % error at $8.78\times$ the speed of Newton–BiCGStab, or 1.9 % at $21.7\times$. At a fixed error, its query time on Burgers barely grows with the mesh (under $2\times$ from 256^2 to 4096^2), while the solver’s grows about $30\times$. At matched latent dimension it is more accurate than a reproduced NM-ROM and POD-LSPG. On three-dimensional Navier–Stokes at 96^3 the nonlinear head reaches 0.15 % at $6.67\times$ the speed of the pseudo-spectral solver that generated the data, and on three-dimensional Burgers at 256^3 the span reaches 2.2 % at $2.2\times$ (3.8 % at $5.6\times$) on sealed held-out cases. On linear Poisson and heat problems in two and three dimensions the ordered span is the best setting, reaching 0.74 % at $106\times$ the speed of conjugate gradients on Poisson at 4096^2 and 0.13 % at $216\times$ that of Crank–Nicolson CG on held-out heat cases at 2048^2 .

1 INTRODUCTION

Two properties decide whether a learned model can stand in for a partial differential equation (PDE) solver: whether accuracy can be traded for speed when the model is used, and how fast the cost of a given accuracy grows as the mesh is refined. Full-order methods (FOMs) solved with iterative solvers such as conjugate gradients (Hestenes and Stiefel, 1952) have the first property, since their tolerance and time step can be set per query, but their cost grows quickly with the mesh. Neural operators (Li et al., 2021; Lu et al., 2021; Kovachki et al., 2023) are fast to evaluate, but at a given mesh each trained model gives one (accuracy, wall-clock) point, and moving it requires retraining. Reduced-order models (ROMs) sit in between: linear projection-based ROMs (Benner et al., 2015; Sirovich, 1987) can require many basis functions for moving fronts (Cohen and DeVore, 2015), and nonlinear-manifold ROMs (NM-ROMs) learn a more compact representation (Lee and Carlberg, 2020; Kim et al., 2022), but their online nonlinear solve can offset the benefit of fewer unknowns. We ask: *can one trained model have both properties, an accuracy choice at query time and a cost that grows slowly with the mesh?*

We answer this question for viscous Burgers in two dimensions and incompressible Navier–Stokes in three, and use the same framework on linear Poisson and heat problems. Our NM-ROM represents the solution by a learned spatial bank multiplied by the output of a small nonlinear head. After training, the bank is ordered once by how much of the training data each of its functions carries. A query then chooses how many of the ordered functions to use, R' , and whether to solve with the nonlinear head or in the linear span of those functions, without changing a learned weight. All online

coefficients come from the weak PDE residual, without access to the reference solution, and stopping tolerances and empirical quadrature set the cost of the solve.

On viscous Burgers one frozen model covers 0.22 % error at $8.78\times$ the speed of Newton–BiCGStab and 1.9 % at $21.7\times$ at 4096^2 (Table 1). From 1024^2 one of its settings is more accurate and faster than the FNO, U-Net, Transolver and DeepONet we trained at each mesh (Table 2; on Burgers these saw fewer training trajectories than our model, §6.2), and at matched latent dimension it is more accurate than a reproduced NM-ROM and POD-LSPG (Table 3). On three-dimensional Navier–Stokes the nonlinear head is the accurate setting, 0.15 % at $6.67\times$ the speed of the pseudo-spectral solver at 96^3 , and at 64^3 and 96^3 it is more accurate and faster than all four operators. On the linear Poisson and heat problems the ordered span is the best setting, up to $106\times$ faster than conjugate gradients on Poisson and $216\times$ faster than Crank–Nicolson CG on held-out heat cases.

Contributions.

1. **Deployment-time tunability from a single trained NM-ROM.** An importance-ordered bank gives nested settings of its width R' , and the choice between the nonlinear head and the linear span sets the accurate end. Accuracy and cost are chosen at query time without retraining, and we measure the full ladders on development and held-out cases.
2. **Accuracy and speed on nonlinear PDEs at large meshes.** On Burgers up to 4096^2 and on three-dimensional Navier–Stokes, the same frozen model is faster than the named full-order solver at the finest meshes, and it is more accurate and faster than the neural operators we trained there and more accurate than prior NM-ROMs. Its empirical quadrature is confirmed on held-out states that the solver reaches.
3. **An architecture and implementation that make this practical.** A coordinate-network bank with exact Dirichlet enforcement, a head with a linear skip, precomputed linear terms and matrix-free JAX evaluation keep the query cost nearly flat as the mesh is refined. At a fixed error our query time grows as about $N^{0.1}$ on Burgers and $N^{0.4}$ on Navier–Stokes, against $N^{0.65}$ and $N^{1.0}$ for the full-order solvers (Figure 2).

2 RELATED WORK

Linear and non-linear manifold ROMs. Linear ROMs approximate solutions in a fixed subspace (Benner et al., 2015; Sirovich, 1987); moving fronts can require a large basis (Cohen and DeVore, 2015; Greif and Urban, 2019). Nonlinear-manifold ROMs combine learned representations with residual projection (Lee and Carlberg, 2020; Kim et al., 2022), including quadratic manifolds (Geelen et al., 2022; Barnett and Farhat, 2022), neural-field representations (Kim et al., 2024) and adaptive bases (Peherstorfer and Willcox, 2015; Carlberg, 2015). Least-squares Petrov–Galerkin projection (Carlberg et al., 2011) and preassembly of reduced operators (Ştefănescu and Sandu, 2014; Weder et al., 2024) are established components. We differ from these by a coordinate-network bank with an exact boundary factor, a head with a linear skip, and a bank ordered after training whose nested widths give deployment-time settings from one trained model, as runtime-tunable networks do (Yu et al., 2019; Cai et al., 2020).

Neural operators and hyper-reduction. FNO (Li et al., 2021), DeepONet (Lu et al., 2021), U-Net (Ronneberger et al., 2015; Takamoto et al., 2022) and Transolver (Wu et al., 2024) learn solution maps. Their evaluation resolution or inference procedure can also change accuracy and cost; our mechanism instead changes the trial space used to solve the PDE. Empirical quadrature (Hernández et al., 2017; Yano and Patera, 2019) reduces nonlinear residual evaluation through non-negative weights. We retain that construction, testing its error on states reached by the solver and through independent re-draws. The main iterative linear-system comparator is conjugate gradient (Hestenes and Stiefel, 1952).

3 METHODOLOGY

To answer whether a single trained model can expose a deployment-time accuracy/speed tradeoff, we turn a learned manifold into a PDE solver through three components. First, we define a trial manifold whose decoder enforces Dirichlet conditions exactly and whose spatial bank is ordered, so

that a query can use fewer of its functions (§3.1). Second, we project the full-order residual onto that manifold against fixed test vectors, which reduces every query to a small least-squares problem (§3.2). Third, we compute every linear term once, offline, and apply Empirical Quadrature to the nonlinear term, so that the cost of a query does not depend on the mesh size (§3.3). The architecture in §3.4 is then designed to support these requirements in practice. Throughout, $u \in \mathbb{R}^N$ denotes the full-order discrete state on a uniform grid and $K \in \mathbb{R}^{N \times N}$ the sparse finite-difference Laplacian.

3.1 TRIAL MANIFOLD WITH EXACT DIRICHLET ENFORCEMENT

We do not use an encoder. The decoder writes the state as a fixed set of spatial functions times coefficients produced by a small network,

$$\tilde{u}(z) = \mathcal{D}(z) = G h(z), \quad G_{x,:} = \mu(x) g_\phi(x)^\top, \quad \mu(x) = 16 x_1(1 - x_1) x_2(1 - x_2). \quad (1)$$

The bank $G \in \mathbb{R}^{N \times R}$ holds R spatial functions, one per column, each a coordinate network g_ϕ read at the grid nodes, and the head $h : \mathbb{R}^k \rightarrow \mathbb{R}^R$ maps the latent state z to their coefficients. Since $\mu = 0$ on the boundary Γ , every column of G vanishes there, so $\partial \tilde{u} / \partial z = 0$ on Γ at any resolution and the boundary equations drop out of the projection. The cube and the L-shaped domain use a factor that vanishes on their boundaries in the same way. A query finds z by solving the PDE (§3.2), starting from the nearest stored training code.

Ordering the bank. Nothing in training orders the columns of G , so its first R' columns are a poor basis. After training we rotate it once,

$$\hat{G} = G T \quad (2)$$

(Appendix A.1). The columns of $\hat{G}$ are orthonormal and ordered by how much of the training data they carry, so the first R' of them are the best R' -dimensional subspace of the bank for the training fields. A query then solves in one of two ways,

$$\tilde{u} = G h(z), \quad z \in \mathbb{R}^k \quad (\text{head}), \quad \tilde{u} = \hat{G}_{R'} c, \quad c \in \mathbb{R}^{R'} \quad (\text{span}), \quad (3)$$

where $\hat{G}_{R'}$ holds the first $R' \leq R$ columns. The head is a nonlinear manifold with k unknowns; the span is a linear space with R' unknowns. The width R' is the accuracy knob. The columns are nested, so narrowing the span needs no retraining, and a narrower span is cheaper both to solve and to turn back into a field.

3.2 PROJECTING THE PDE ONTO THE MANIFOLD

Each PDE we consider gives a discretised residual $R(u)$ that vanishes at the discrete solution. We substitute the trial manifold into R and, instead of projecting with the decoder Jacobian, test it against M fixed vectors, the rows of $P \in \mathbb{R}^{M \times N}$. On the square these are the M lowest sine modes, which are eigenvectors of K . A query solves

$$z^* = \arg \min_z \frac{1}{2} \|P R(\tilde{u}(z))\|_2^2, \quad M > k, \quad (4)$$

and the span the same problem in c . This is least-squares Petrov–Galerkin projection (Carlberg et al., 2011; Lee and Carlberg, 2020); each row of PR is also scaled by a fixed factor that puts the tests on a common scale (Appendix A). We solve (4) by damped Gauss–Newton (Levenberg–Marquardt) (Marquardt, 1963),

$$\Delta z = -(J^\top J + \lambda \text{diag}(J^\top J))^{-1} J^\top P R(\tilde{u}(z)), \quad J = P \frac{\partial R}{\partial u} J_{\mathcal{D}}(z), \quad (5)$$

and keep a step only if the residual decreases. Here $J_{\mathcal{D}}(z) = G J_h(z)$ for the head and $J_{\mathcal{D}} = \hat{G}_{R'}$, a constant, for the span. Unlike $J_{\mathcal{D}}(z)^\top$, the tests P do not change with z , which is what lets every linear term be computed once (§3.3). For Poisson and heat in the span, (4) is a linear least-squares problem solved once; the iteration is needed only for the head and for the nonlinear PDEs.

Elliptic (Poisson). The Poisson residual is $R(u) = Ku - F$. In the span it is linear in c ,

$$P R(\hat{G}_{R'} c) = (PK \hat{G}_{R'}) c - PF, \quad (6)$$

so a query is one least-squares solve with a fixed $M \times R'$ matrix, factorised once per mesh and width, and nothing is iterated. With the head, (5) iterates in z with $J = PKG J_h(z)$, and only the head is re-evaluated.

Parabolic (heat). The semi-discretisation $du/dt = -\kappa Ku$ is advanced by Crank–Nicolson, with the residual

$$R(u_{n+1}) = \left(I + \frac{\Delta t \kappa}{2} K\right) u_{n+1} - \left(I - \frac{\Delta t \kappa}{2} K\right) u_n. \quad (7)$$

Each step solves

$$z_{n+1} = \arg \min_z \|P R(\tilde{u}(z))\|_2, \quad u_n = \tilde{u}(z_n), \quad (8)$$

by (5), starting from z_n , and the span the same in c_{n+1} . In the span the step is linear in c_{n+1} , so the rollout applies one fixed $R' \times R'$ matrix per step.

Hyperbolic (Burgers). For $u_t + u(u_x + u_y) = \nu \Delta u$, with backward Euler and a sign-upwind stencil (Appendix A.4), the residual is

$$R(u_{n+1}) = u_{n+1} - u_n + \Delta t (A(u_{n+1}) + \nu K u_{n+1}), \quad (9)$$

where $A(u)$ is the advection term. The time derivative and diffusion are linear in the state; advection is not, so both the head and the span solve each step (8) by iterating (5), starting from the previous step.

Incompressible flow (Navier–Stokes). For $u_t + (u \cdot \nabla)u = -\nabla p + \nu \Delta u$, $\nabla \cdot u = 0$, on the periodic cube, with the implicit midpoint rule, the residual is

$$R(u_{n+1}) = u_{n+1} - u_n + \Delta t ((\bar{u} \cdot \nabla) \bar{u} + \nabla \bar{p} - \nu \Delta \bar{u}), \quad \bar{u} = \frac{1}{2}(u_n + u_{n+1}). \quad (10)$$

The tests P are divergence-free Fourier modes, so the pressure drops out of PR . Advection is quadratic in the state, so in the bank it is a fixed quadratic form of the coefficients, computed once, and needs no quadrature. Both the head and the span solve each step (8) by Gauss–Newton (5). The vortices travel across the domain, so the bank is centred on the flow, and each step also solves for the three-number shift of that centre (Appendix A.5).

3.3 HYPER-REDUCTION

The manifold reduces the unknowns from N to k or R' , but the tested residual $PR(\tilde{u})$ still visits every node, so each evaluation costs $O(NR)$, which kills the wall-clock benefit. We remove this cost in two ways, one for linear terms and one for the nonlinear term.

Linear terms: computed once and exact. A linear term is tested once, offline. For the Laplacian,

$$PK\tilde{u} = (PKG) h(z), \quad PKG \in \mathbb{R}^{M \times R}, \quad (11)$$

and the span uses $PK\hat{G}_{R'}$, the first R' columns of $PKGT$. Online a linear term costs $O(MR)$ and is exact. For Poisson and heat this is the whole residual, so their reduced solves make no approximation beyond the trial space. The quadratic term of Navier–Stokes is also tested exactly, as a precomputed tensor.

The nonlinear term: Empirical Quadrature. Burgers advection cannot be computed once. We use Empirical Quadrature (Hernández et al., 2017; Yano and Patera, 2019) to select a non-negative per-node weight vector w , independent of the state, such that

$$PA(\tilde{u}) = \sum_{i=1}^N P_{:,i} A_i(\tilde{u}) \approx \sum_{i \in \mathcal{S}} w_i P_{:,i} A_i(\tilde{u}). \quad (12)$$

Letting $g_i^{(\ell)} = P_{:,i} A_i(\tilde{u}^{(\ell)}) \in \mathbb{R}^M$ denote the per-node contribution at a stored state $\tilde{u}^{(\ell)}$ that the solver reaches on training trajectories, the constraint at that state is $\sum_i w_i g_i^{(\ell)} = PA(\tilde{u}^{(\ell)})$. Stacking the constraints gives a matrix $\mathcal{G}$ and a vector b , and we solve

$$\min_{w \geq 0} \|\mathcal{G} w - b\|_2 \quad (13)$$

by NNLS (Lawson and Hanson, 1974), which returns a sparse w with support $\mathcal{S}$ of size $N_{\text{eq}} \ll N$. Computing A_i needs $\tilde{u}$ at node i and its stencil neighbours $\mathcal{N}(i)$, so we evaluate the decoder only on $\mathcal{S}^+ = \mathcal{S} \cup \bigcup_{i \in \mathcal{S}} \mathcal{N}(i)$, with $|\mathcal{S}^+| \leq 5N_{\text{eq}}$. Because each bank column is a coordinate network (§3.4), those rows of G are computed once and cached.

Cost. With linear terms computed once and advection on $\mathcal{S}^+$, one iteration of (5) costs

$$\mathcal{O}(MR(N_{\text{eq}} + k)), \quad (14)$$

with k replaced by R' in the span, independent of N . Returning the field at an output time is one further $O(NR')$ product.

Checking the rule. A rule is an approximation, so we check it where it is used: on held-out states that the reduced solver actually reaches. A rule is confirmed when the relative error of the tested advection, ρ in (20), stays below a bar fixed in advance on every held-out draw and on one further confirmation draw (Appendix C.1). Every rule behind Table 1 is confirmed. Backward Euler never evaluates advection at the initial state. Our check as first registered included that state; we removed it after seeing that it was the only state at which the widest span settings failed, and this is the one change we made to the check after seeing results. The rule changes how the residual is evaluated, not what the model can represent, so it is a cost knob, separate from R' .

3.4 MODEL ARCHITECTURE

Three properties of the solve motivate the model in Figure 1. First, every head solve starts from a stored training code and descends along the head’s Jacobian, so that Jacobian must keep a usable direction; this motivates a *linear skip* in the head. Second, empirical quadrature keeps a sparse set of nodes and discards the rest, which saves time only if the decoder can be evaluated at one node without the others, and one trained model must serve every mesh of a series; this motivates a *coordinate-network bank*. Third, one trained model must offer several accuracy settings without retraining; this motivates an *ordered bank*. Widths and training settings are in Appendix C.

Linear skip, for the latent solve. The head is a shallow network plus a linear map,

$$h(z) = W_{\text{dir}}z + \varphi_{\theta}(z), \quad \varphi_{\theta}(z) = W_3 \sigma(W_2 \sigma(W_1 z + b_1) + b_2) + b_3, \quad (15)$$

with σ the SiLU activation. Its Jacobian is $J_h(z) = W_{\text{dir}} + J_{\varphi}(z)$. The first term does not depend on z , so the Levenberg–Marquardt step from a stored code always has a fixed linear direction to follow, while φ_{θ} supplies the curvature of the manifold. The head acts on the R bank coefficients, not on the n nodes, so its cost and its parameter count do not depend on the mesh.

Coordinate-network bank, for node-local evaluation. Each column of the bank is a coordinate network read at the node and multiplied by the boundary factor of (1),

$$G_{x,r} = \mu(x) g_{\phi,r}(x), \quad g_{\phi}(x) = \text{MLP}_{\phi}(\gamma(x)), \quad \gamma(x) = [\cos(2\pi\Omega x), \sin(2\pi\Omega x)], \quad (16)$$

where γ is a random-Fourier-feature embedding with a fixed random frequency matrix Ω (Tancik et al., 2020). Because g_{ϕ} reads a coordinate, a row of G is computed from its node alone. A quadrature rule therefore needs only the rows of $\mathcal{S}^+$, and the same trained network is sampled on every mesh. Two natural alternatives fail one of these tests. A convolutional or attention decoder couples its output nodes, so one node costs a pass over the whole grid, and a learned dense readout stores NR parameters and must be retrained for each mesh. The parameter count of g_{ϕ} does not depend on n .

Ordered bank, for deployment-time accuracy. The rotation (2) puts the most useful directions first. Cutting the bank at R' then gives the best R' -dimensional subspace of the bank for the training fields in the least-squares sense, and the settings are nested, so one model built with R columns contains every smaller one. A query picks R' and no weight changes. The two solve modes of (3) are the two ends of the model. The span is a linear ROM in a learned bank, and on a linear PDE it is one least-squares solve. The head is a nonlinear-manifold ROM in k unknowns and can follow states that no small span contains. The alternatives, a separate model per accuracy or a retrained smaller bank, both need retraining. One frozen model supplies both settings of every row of Table 1.

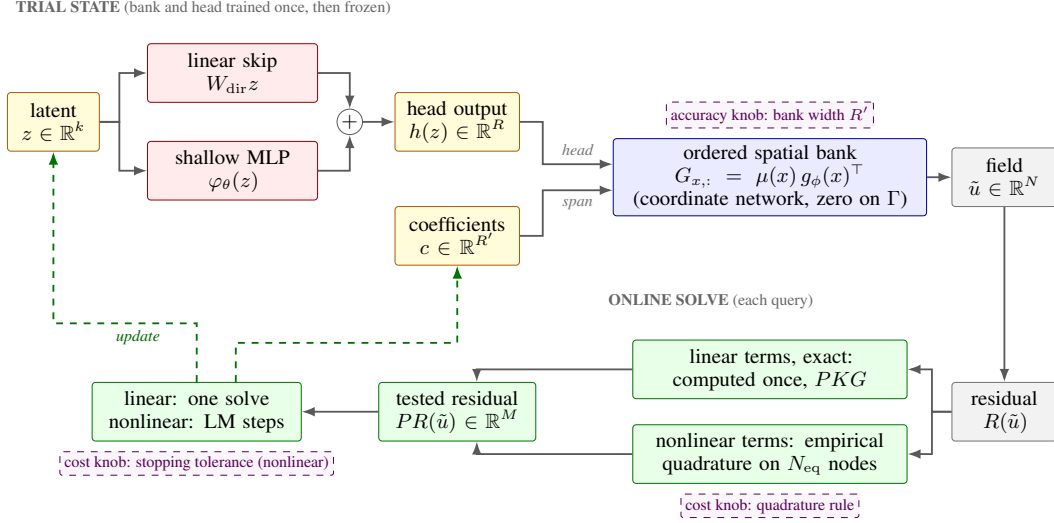


Figure 1: NM-ROM model and online solve. Top, trained once and then frozen: the latent code z passes through a linear skip and a shallow SiLU network, and their sum gives the bank coefficients $h(z)$. The ordered coordinate-network bank, multiplied by a factor that vanishes on the boundary Γ , turns coefficients into the field $\tilde{u}$ with the Dirichlet data exact. A query uses either the head, $\tilde{u} = Gh(z)$, or the span of the first R' ordered columns, $\tilde{u} = \hat{G}_{R'} c$. **Bottom, solved at each query:** the residual of $\tilde{u}$ is tested against M fixed tests P . Linear terms use the matrix PKG , computed once, and are exact; Burgers advection uses the empirical quadrature rule on N_{eq} nodes. On a linear PDE in the span the tested residual is linear in c and is solved once by least squares; otherwise Levenberg–Marquardt steps update z or c until the stopping test is met. Dashed tags mark the deployment knobs: the bank width R' sets the accuracy, and the quadrature rule and the stopping tolerance set the cost.

What is fixed and what is solved. Training fixes the bank, its ordering, the head and the tests for every row of Table 1. Each query solves z or c . Deployment chooses the mode and R' , the test count M , the quadrature and the stopping rule, and leaves the weights fixed.

4 IMPLEMENTATION

4.1 MATRIX-FREE PROJECTED OPERATORS VIA JAX

The matrix PKG of §3.3 is built once per mesh, so the online value of a linear term is one small matrix–vector product. In the span the Jacobian is the fixed matrix $PKG\hat{G}_{R'}$. With the head the remaining derivative is the Jacobian of the head, which forward-mode automatic differentiation (Bradbury et al., 2018) supplies,

$$J_h v = \text{JVP}(h, z, v), \quad J = PKG J_h(z). \quad (17)$$

Each accepted step solves the damped normal equation directly. A linear PDE does not return to the grid, and the projected operator is not passed to a Krylov method. Burgers advection reads the cached stencil on the quadrature nodes, and its Jacobian is assembled analytically (Appendix A.4). The ordered bank is stored in column blocks at the tested widths, so a query at width R' reads exactly R' columns.

4.2 TRAINING PROTOCOL

Training is an auto-decoder on training data only, in two stages. The coordinate network g_ϕ is fitted to the training states through (1) and frozen, which fixes the bank. The head is then fitted to the bank coefficients of those states, with one stored code per training state. Last, the bank is ordered once by (2). This step has no trainable weights and takes one QR and one SVD. Widths, cohorts and offline costs are in Appendix C.

5 EXPERIMENTAL SETUP AND BENCHMARK PROBLEMS

Problems and references. In two dimensions we solve Poisson on the square, heat and viscous Burgers, with an L-shaped Poisson domain as an ill-conditioned check. In three dimensions we solve Poisson, heat, viscous Burgers and incompressible Navier–Stokes on the periodic cube, whose initial velocity is the curl of two localised vector potentials with random centre, separation, strength and viscosity. Meshes and cohorts are in Table C.2, the sampled families in Table C.3 and the three-dimensional configurations in Table C.7.

Accuracy. We report the worst relative L^2 error over the stated cases and requested times, in percent. Poisson uses the steady reference norm and heat the current reference norm; Burgers and Navier–Stokes use the initial-field norm over evolved times. Same-grid error measures the reduction error against the converged discrete solution. Error against a refined reference also includes discretisation error, and we never interchange the two. Settings for a held-out cohort are fixed before its cases are opened; all other cohorts are development cohorts.

Timing and speedup. Every accuracy–runtime pair comes from the same solver invocation, in double precision, with GPU burn-in, synchronised timing and medians of retained repetitions (protocol in Appendix C.1). GPU query time covers initialisation, the solve or rollout and the requested device outputs. Training and compilation are offline costs. Speedup is $S = T_{\text{FOM}}/T_{\text{method}}$ with both times from the same allocation, and each table names the FOM and shows its error. Solves that miss their stopping rule are reported as such and do not enter a speedup.

Baselines. Poisson, including the L-shaped check, is compared with conjugate gradients (CG), and heat with Crank–Nicolson stepping whose linear solve is CG (CN–CG). Burgers is compared with Newton–BiCGStab. Navier–Stokes is compared with the solver that generated its data, a Fourier pseudo-spectral method with Crank–Nicolson diffusion and Adams–Bashforth advection (CNAB2), which needs no linear solve. The named FOM of a row is that solver at its fastest tested setting that is at least as accurate as the accurate NM-ROM setting. Other nonlinear-manifold ROMs are compared in Table 3, and neural operators (FNO, U-Net, DeepONet and Transolver) in §6.2.

6 NUMERICAL EXPERIMENTS AND RESULTS

Table 1 compares the NM-ROM with the named full-order solvers at every problem and mesh, and Table 2 with neural operators. The subsections that follow look at the bank-width knob behind the two settings of each row, and at which knob to turn.

6.1 ACCURACY AND SPEED AGAINST FULL-ORDER SOLVERS

Linear problems. On Poisson both settings of a row are solves in the span of the ordered bank. At 4096^2 the accurate width $R' = 512$ reaches 0.74 % at $106\times$ the speed of CG, and the fast width $R' = 128$ reaches 2.31 % at $358\times$. The head is less accurate here (3.15 %): on a linear problem the best solve in the bank is the linear one, and it is also the cheapest. The speedup grows with the mesh because the reduced query does not visit the grid until it returns the field. On the L-shaped check at 2048^2 the accurate setting is $84.2\times$ faster than CG at 3.03 %. A wide heat bank, evaluated on a held-out cohort never used for selection, reaches 0.13 % over all times in the span of its first 128 ordered functions and is $216\times$ faster than CN–CG at 2048^2 . In three dimensions the held-out Poisson cohort gives 0.23 % at $8.07\times$ at 64^3 . The three-dimensional heat model, evaluated on a held-out cohort of 64 cases, reaches 0.083 % over all times in the span of its first 320 ordered functions and is $7.60\times$ faster than CN–CG at 256^3 ; the span of 128 reaches 1.11 % at $16.8\times$. As on Poisson, the span is both more accurate and cheaper than the head at every mesh.

Burgers. At coarse meshes the reduced solve is slower than Newton–BiCGStab, but its cost barely changes with the mesh. The fast setting (the span of the first 128 ordered functions) overtakes the FOM by 512^2 ($1.59\times$ at 1.75 %) and is $21.7\times$ faster at 4096^2 . The accurate setting (the span of 384) reaches 0.17–0.22 % at every mesh and is faster than the FOM from 1024^2 , $8.78\times$ at 4096^2 . Every quadrature rule in these rows is confirmed on the states its residual evaluates (§3.3). On 64 held-out

Table 1: NM-ROM against the named full-order solver at every problem and mesh. **Both NM-ROM columns of a row come from one frozen model:** *accurate* is the most accurate tested setting and *fast* the cheapest setting at least as accurate as the model’s earlier default (Table C.4); no network is retrained between them. Error is the worst relative L^2 error over the cases (%). Speedup is the FOM’s median time divided by the NM-ROM’s in the same GPU allocation, **bold** where the NM-ROM is faster. The FOM is the named solver at its fastest tested setting at least as accurate as the accurate setting, and both speedups of a row divide that one time. The FOM is CG for Poisson, Crank–Nicolson with CG (CN–CG) for heat, Newton–BiCGStab for Burgers and the pseudo-spectral CNAB2 for Navier–Stokes; its tolerance in each row is in Table C.4. The Navier–Stokes and three-dimensional Burgers fast settings are the cheapest bank width below 5 % error; the three-dimensional Burgers rows are sealed held-out cases. Dashes mark settings not measured.

Problem	Mesh	NM-ROM accurate		NM-ROM fast		FOM
		Err. (%)	Speedup	Err. (%)	Speedup	Err. (%)
<i>Two-dimensional</i>						
Poisson	256 ²	0.75	13.4 ×	2.31	17.5 ×	0.55
	1024 ²	0.74	27.9 ×	2.31	68.5 ×	0.22
	2048 ²	0.74	54.1 ×	2.31	174 ×	0.61
	4096 ²	0.74	106 ×	2.31	358 ×	0.45
Poisson L-shape	256 ²	3.04	47.1 ×	5.73	53.4 ×	2.40
	512 ²	3.03	39.4 ×	5.72	45.5 ×	1.51
	1024 ²	3.03	48.9 ×	5.72	57.9 ×	1.05
	2048 ²	3.03	84.2 ×	5.72	104 ×	0.76
Heat	1024 ²	0.13	105 ×	0.28	171 ×	0.045
	2048 ²	0.13	216 ×	0.28	405 ×	0.045
Burgers	4096 ²	0.13	231 ×	0.28	410 ×	0.054
	256 ²	0.17	0.37×	1.60	0.97×	0.048
	512 ²	0.19	0.60×	1.75	1.59 ×	0.050
	1024 ²	0.21	1.46 ×	1.83	3.39 ×	0.048
	2048 ²	0.22	3.30 ×	1.87	8.46 ×	0.049
4096 ²	0.22	8.78 ×	1.89	21.7 ×	0.050	
<i>Three-dimensional</i>						
Poisson	32 ³	0.23	4.42 ×	0.48	4.91 ×	0.16
	64 ³	0.23	8.07 ×	0.47	10.1 ×	0.11
	128 ³	0.14	13.7 ×	0.24	21.5 ×	0.075
	256 ³	0.14	23.2 ×	0.24	40.6 ×	0.049
Heat	32 ³	0.080	10.9 ×	1.11	15.2 ×	0.078
	64 ³	0.082	8.50 ×	1.11	14.3 ×	0.081
	128 ³	0.083	5.79 ×	1.11	12.7 ×	0.082
	256 ³	0.083	7.60 ×	1.11	16.8 ×	0.082
Burgers	64 ³	1.01	0.21×	3.37	2.08 ×	0.92
	128 ³	1.46	0.57×	4.26	5.10 ×	1.39
	256 ³	2.20	2.19 ×	3.77	5.60 ×	1.52
Navier–Stokes	32 ³	0.15	1.09 ×	2.96	1.87 ×	0.094
	64 ³	0.15	2.89 ×	2.96	5.09 ×	0.093
	96 ³	0.15	6.67 ×	2.96	11.6 ×	0.063

cases the same settings reach a worst error of 0.88–1.35 % (median 0.03 %) for the accurate setting and 2.55–3.11 % (median 0.27–0.30 %) for the fast one at 256²–1024² and 4096², at 0.38–10.6× and 0.94–27.3× the speed of Newton–BiCGStab in the same jobs: a few unseen cases are harder than the development cases, while the typical case is not.

In three dimensions one frozen span model, trained on 32³–128³ fields and used unchanged at 256³, is tested on 32 sealed held-out cases with settings chosen on 64 validation cases. At 256³ the span of 512 reaches 2.20 % at 2.19× the speed of Newton–BiCGStab and the span of 256 reaches 3.77 % at 5.60×. The fast setting is also faster at 64³ and 128³ (2.08× and 5.10×), but there the accurate setting is slower than the FOM, whose cost only grows sharply beyond 128³. The advection term here is a precomputed quadratic form of the coefficients, confirmed on reached states like the quadrature rules ($\rho \leq 2 \times 10^{-3}$). The nonlinear head did not generalise on this family and is not used.

Navier–Stokes. Here the head earns its place. At 96^3 the head with $k = 8$ reaches 0.15 % and is $6.67\times$ faster than CNAB2 at matched accuracy. The span of the same bank is $36\times$ less accurate at the same number of unknowns, and even the full rank-64 span reaches only 0.62 %. The moving vortices lie on a low-dimensional curved manifold that the head follows and a small linear span does not. The error stays flat from 32^3 to 96^3 while the speedup rises from $1.09\times$ to $6.67\times$. On 32 held-out cases the frozen settings give 0.21 % at $7.79\times$ ($8.01\times$ when re-timed alongside the operators, Table 2).

Where the speedup comes from. On the linear problems the query does not iterate. In the span the tested residual is linear in c , so the query is one least-squares solve with a matrix factorised offline, and for heat each time step is a fixed matrix applied to c . The query time is then almost entirely reading the input and writing the output field, which costs $O(NR')$: on Poisson at 4096^2 , rebuilding the field takes 41.8 of the 42.4 ms, while CG needs about 5300 iterations over all nodes. On the nonlinear problems the query iterates, but the count does not grow with the mesh. Burgers takes about one Levenberg–Marquardt iteration per time step (54 over 50 steps for the accurate setting at every mesh from 1024^2 to 4096^2), and Navier–Stokes a fixed three Gauss–Newton sweeps per step. Each iteration costs (14), independent of N , while the full-order solvers sweep all N nodes in every sparse matrix–vector product and, for CG, need more of them as the mesh is refined (Saad, 2003). For a fixed error this is plain in Figure 2: refining the Burgers mesh 256-fold raises our query time less than twofold and the solver’s about thirtyfold, and above 1024^2 our model is cheaper at every target. Fitting time $\propto N^p$ at a fixed error gives $p \approx 0.1$ for our model on Burgers and 0.4 on Navier–Stokes, against about 0.65 and 1.0 for the full-order solvers; on heat, where only reading and writing the field grows, $p \approx 0.9$ against 1.2. So in six of the eight series of Table 1 (Poisson, heat and Burgers in two dimensions, Poisson, Burgers and Navier–Stokes in three) the speedup rises at every refinement while the error stays near its coarse-mesh value (Table C.5). On the L-shape it dips at 512^2 and then rises to $84.2\times$ at 2048^2 ; on three-dimensional heat it stays between $5.79\times$ and $10.9\times$ for the accurate setting (12.7–16.8 $\times$ fast) at 0.080–0.083 % error.

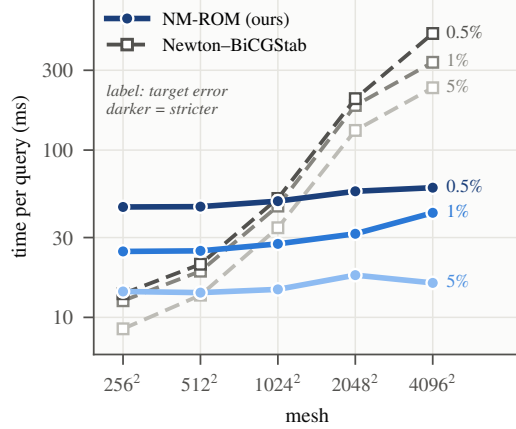


Figure 2: Time to reach a target error on two-dimensional Burgers (6 development cases). Each point is the cheapest setting that meets the target: our model chooses its bank width, Newton–BiCGStab its time step and tolerance. Darker lines mean a stricter target. Fitting time $\propto N^p$ over 256^2 – 2048^2 (one GPU type) gives $p \approx 0.05$ for our model and $p \approx 0.65$ for the solver; up to 4096^2 our time grows less than twofold, the solver’s about thirtyfold.

6.2 COMPARISON WITH NEURAL OPERATORS AND OTHER ROMS

We train an FNO (Li et al., 2021), a U-Net (Ronneberger et al., 2015), a DeepONet (Lu et al., 2021) and a Transolver (Wu et al., 2024) at the mesh it is evaluated on, each with AdamW, one fixed seed and a 3000 s wall budget per network, and time them with the NM-ROM and the full-order solver in one allocation per problem (Table 2). On heat and Navier–Stokes the operators use our model’s own training trajectories and validation split; on Burgers they use 128 trajectories of the same family (Appendix B). Our Burgers and heat models were trained once, at 256^2 and 64^2 , and are used unchanged at every mesh, while each operator was trained at the mesh it is evaluated on. Each trained operator gives one fixed operating point.

The NM-ROM Pareto-dominates every operator at the larger meshes. In every two-dimensional cell from 1024^2 up, and on Navier–Stokes at 64^3 and 96^3 , one NM-ROM setting is more accurate and faster than all four operators (Table 2, bold). On Burgers the head with $k = 16$ reaches 2.29 % at $2.02\times$ the speed of Newton–BiCGStab at 1024^2 and 2.37 % at $4.55\times$ at 2048^2 ; the operators reach 4.60–38.3 % there, and none is faster than $1.46\times$. On heat the accurate setting reaches 0.13 % at $87.4\times$ the speed of CN–CG at 1024^2 and $289\times$ at 2048^2 , against 1.56–10.9 % for the operators. On Navier–Stokes the accurate setting reaches 0.15 % at $3.05\times$ the speed of CNAB2 at 64^3 , and 0.21 %

Table 2: Best NM-ROM against FNO, U-Net, Transolver and DeepONet at every measured (PDE, N) cell. NM-ROM *accurate* and *fast* come from the same trained model. Error is the worst same-grid relative L^2 error (%), over evolved times for Burgers and all times for heat. Speedup is the median GPU time of one full-order setting per cell, the fastest tested one at least as accurate as the NM-ROM accurate setting, divided by the method’s time in the same allocation; *italic* marks speedups below $1\times$. **Bold** marks an NM-ROM setting that is both more accurate and faster than every operator in the cell. Each operator family shows its most accurate trained size. Cells: Burgers 256^2 , job 4181372, six development cases, FOM Newton–BiCGStab tol 10^{-3} , $\Delta t=0.005$ (0.049 %); Burgers 1024^2 and 2048^2 , jobs 4204019 and 4218390, the same six cases, FOM tol 3×10^{-3} , $\Delta t=0.005$ (0.048 and 0.049 %); heat 1024^2 and 2048^2 , jobs 4206383 (A100) and 4211204 (H200), the 16 held-out cases of Table 1, FOM CN–CG $\Delta t=0.025$, rtol 10^{-4} (0.045 %); Navier–Stokes 32^3 and 64^3 , jobs 4234771 and 4237885, the 16 development cases of Table 1, FOM CNAB2 with 50 steps (0.094 and 0.093 %); Navier–Stokes 96^3 , job 4244423, 32 held-out cases, FOM CNAB2 with 70 steps (0.056 %). In these jobs the Burgers accurate setting (span, $R' = 384$, the same quadrature rule as in Table 1, errors equal to four digits) was solved by the general Levenberg–Marquardt code (Jacobian by forward-mode differentiation, dense LU solve, linear predictor) instead of the fused solver of Table 1, whose times that table gives; it was not timed at 256^2 , and the 2048^2 cell ran on an H200 GPU; the Burgers fast column is the head with $k = 16$; the heat fast setting was not timed in its job.

PDE	N	NM-ROM accurate		NM-ROM fast		FNO		U-Net		Transolver		DeepONet	
		Err. (%)	Speedup	Err. (%)	Speedup	Err. (%)	Speedup	Err. (%)	Speedup	Err. (%)	Speedup	Err. (%)	Speedup
Burgers 2D	256^2	—	—	1.89	<i>0.81</i> $\times$	7.42	<i>4.21</i> $\times$	4.49	<i>3.08</i> $\times$	4.46	<i>2.81</i> $\times$	32.5	<i>8.37</i> $\times$
	1024^2	0.21	<i>0.22</i> $\times$	2.29	2.02 $\times$	5.69	<i>0.70</i> $\times$	4.60	<i>1.31</i> $\times$	10.2	<i>0.52</i> $\times$	32.8	<i>1.36</i> $\times$
	2048^2	0.22	<i>0.66</i> $\times$	2.37	4.55 $\times$	8.02	<i>0.27</i> $\times$	9.50	<i>0.81</i> $\times$	18.7	<i>0.86</i> $\times$	38.3	<i>1.46</i> $\times$
Heat 2D	1024^2	0.13	87.4 $\times$	—	—	3.59	<i>1.07</i> $\times$	1.56	<i>1.93</i> $\times$	2.57	<i>6.22</i> $\times$	4.60	<i>2.09</i> $\times$
	2048^2	0.13	289 $\times$	—	—	3.98	<i>0.93</i> $\times$	2.76	<i>2.76</i> $\times$	3.66	<i>40.5</i> $\times$	10.9	<i>5.11</i> $\times$
Navier–Stokes 3D	32^3	0.15	<i>1.17</i> $\times$	2.96	<i>2.10</i> $\times$	0.40	<i>2.16</i> $\times$	0.48	<i>0.45</i> $\times$	0.54	<i>0.42</i> $\times$	52.6	<i>1.84</i> $\times$
	64^3	0.15	3.05 $\times$	2.96	<i>5.40</i> $\times$	1.34	<i>2.05</i> $\times$	1.11	<i>0.74</i> $\times$	1.22	<i>0.70</i> $\times$	51.7	<i>1.36</i> $\times$
	96^3	0.21	8.01 $\times$	3.24	<i>14.4</i> $\times$	1.51	<i>3.23</i> $\times$	2.29	<i>1.44</i> $\times$	1.16	<i>6.65</i> $\times$	51.4	<i>2.26</i> $\times$

Table 3: Best NM-ROM against other nonlinear-manifold and projection ROMs on Burgers 2D at matched latent dimension ($k = 16$ for every baseline and for our head). Error is the worst evolved same-grid relative L^2 error (%). Speedup as in Table 2: one full-order setting per cell, the fastest at least as accurate as the NM-ROM accurate setting, same allocation; *italic* below $1\times$; **bold** marks an NM-ROM setting more accurate and faster than every baseline. At 256^2 and 512^2 the cohort is 32 validation cases (never used for any choice) and every method, ours included, uses a dense residual, so no times are given; at 1024^2 and 2048^2 it is the six development cases of jobs 4204019 and 4218390, where our accurate setting is timed with the general solver code, as in Table 2. The shallow masked-autoencoder NM-LSPG of Kim et al. (2022) passed its reproduction gate on the third of three pre-registered attempts; its hyper-reduction was not reproduced and it was not run from 1024^2 . The quadratic manifold (Geelen et al., 2022) was run at 1024^2 and 2048^2 only. Every latent size and the memory use are in Tables C.6 and B.1.

PDE	N	NM-ROM accurate		NM-ROM head, $k=16$		Kim et al. NM-LSPG		POD-LSPG		Quadratic manifold	
		Err. (%)	Speedup	Err. (%)	Speedup	Err. (%)	Speedup	Err. (%)	Speedup	Err. (%)	Speedup
Burgers 2D	256^2	—	—	6.79	—	145.40	—	41.53	—	—	—
	512^2	—	—	7.72	—	174.99	—	42.01	—	—	—
	1024^2	0.21	<i>0.22</i> $\times$	2.29	2.02 $\times$	—	—	29.3	<i>0.40</i> $\times$	22.6	<i>0.096</i> $\times$
	2048^2	0.22	<i>0.66</i> $\times$	2.37	4.55 $\times$	—	—	29.4	<i>0.42</i> $\times$	22.7	<i>0.094</i> $\times$

at $8.01\times$ on held-out cases at 96^3 , against 1.16–51.7 % for the operators, none faster than $6.65\times$. The exceptions are the small meshes. On Burgers at 256^2 the operators are faster than the NM-ROM, at 4.5–33 % error against its 1.89 %, and on Navier–Stokes at 32^3 the FNO (0.40 % at $2.16\times$) is faster than our accurate setting and more accurate than our fast one. As in Table 1, the NM-ROM’s lead grows with the mesh, because its query cost barely changes while the operators and the full-order solver both pay for every node. Each trained operator gives one fixed point, and a different accuracy needs a new training run.

The NM-ROM is more accurate than every other ROM at matched size. At $k = 16$ our head reaches 6.79–7.72 % at 256^2 and 512^2 , against 145–175 % for the reproduced shallow masked-autoencoder NM-ROM and 42 % for POD-LSPG (Table 3). At 1024^2 it reaches 2.29 % at $2.02\times$ the speed of Newton–BiCGStab, and at 2048^2 2.37 % at $4.55\times$, against 29.3–29.4 % for POD-LSPG and 22.6–22.7 % for the quadratic manifold, both slower than the full-order solver. POD-LSPG

approaches 1 % only with 256 or more unknowns, and is then two orders of magnitude slower (Table B.1).

6.3 WHICH KNOB TO TURN

Every measurement behind this section, including the full ladders, is in Table D.1 in the appendix. None of these settings retrains a weight.

Accuracy: the bank width and the head. The bank width R' changes what the trained model can represent, so it is the accuracy knob. On Poisson at 4096^2 , narrowing the bank from $R' = 512$ to 32 raises the error $12.4\times$, from 0.74 to 9.17 %, and lowers the query time $9.65\times$, so the same model moves from $106\times$ to $1022\times$ the speed of CG. On heat at 2048^2 the span runs from 0.13 % at $216\times$ ($R' = 128$) to 1.08 % at $512\times$ ($R' = 32$), and on Burgers at 4096^2 from 0.22 % at $8.78\times$ ($R' = 384$) to 10.7 % at $43.2\times$ ($R' = 32$). On Navier–Stokes the span runs from 0.62 % at $R' = 64$ to 5.55 % at $R' = 8$, at $1.89\times$ lower cost. The choice between head and span depends on the problem. On Poisson and heat the span wins at every width, so the knob is R' alone. On Navier–Stokes the head is the accurate end: at the same number of unknowns it is $36\times$ more accurate than the span, and costs about twice as much, because each step also evaluates the network and its Jacobian. This is what makes the NM-ROM tunable in a way a trained neural operator is not: the same frozen model is reused, and only a deployment argument changes.

Limitations. (i) Except for Navier–Stokes, speedups are against named iterative full-order solvers at the stated tolerances. Fast direct and spectral solvers exist for the square and the cube and would be faster than CG there. The named FOM is more accurate than the NM-ROM in every row. (ii) The Navier–Stokes bank is a POD basis in a moving frame, not a coordinate network, and the family has moderate Reynolds numbers; we make no claim about turbulent flow. (iii) On Poisson the errors lie near the bank floor (0.742 % square, 0.143 % cube), so a wider or better bank, not the solver, limits accuracy. On the L-shape (3.03 % against a full-bank floor of 0.77 %) the span cannot be wider than 128 with the available tests, so more tests, not the bank, limit accuracy. (iv) Quadrature validation covers finitely many reached states and is not a global certificate, and we changed the check once after seeing results (§3.3); no rule is used in three dimensions. Reduced solves can converge to a wrong branch, and lower same-grid error does not remove discretisation error. (v) The evaluation cohorts are small (6–64 cases). Timings report medians without dispersion.

7 CONCLUSION AND FUTURE WORK

We built an NM-ROM whose distinguishing property is a deployment-time accuracy/speed tradeoff from a single trained model: at inference, the width of the ordered bank, the choice between the head and the span, and on the nonlinear problems the quadrature rule and the stopping tolerance trade accuracy for speed. This is the lever neural operators do not expose, since each trained FNO or DeepONet delivers one fixed operating point at a given mesh. Mechanically, the decoder is a learned manifold built from a coordinate-network bank and a nonlinear head; the PDE itself is enforced by least-squares Petrov–Galerkin projection of the discrete residual onto that manifold; Dirichlet boundary conditions are enforced exactly by the bank; linear terms are preassembled exactly and empirical quadrature removes the mesh from the nonlinear term; and the whole pipeline is matrix-free in JAX.

Three pieces had to come together to make this work. First, a coordinate-network bank, which is evaluated at one node without the others and serves every mesh of a series from one trained network. Second, a one-time ordering of that bank, which turns one trained model into a nested family of trial spaces chosen at the query. Third, a head with a linear skip, which gives the latent solve a usable direction from a stored code and supplies the accurate end where a linear span is not enough, as on Navier–Stokes.

The model keeps its accuracy as the mesh is refined, so its speedup over the named solvers grows with resolution: at a fixed error its query time grows as about $N^{0.1}$ on Burgers and $N^{0.4}$ on Navier–Stokes, against $N^{0.65}$ and $N^{1.0}$ for the solvers. On viscous Burgers at 4096^2 it reaches 0.22 % at $8.78\times$ the speed of Newton–BiCGStab, and on three-dimensional Navier–Stokes at 96^3 the head reaches 0.15 % at $6.67\times$ the speed of a spectral solver, $36\times$ more accurate than a linear span of the same size.

On the linear problems the ordered span is enough: 0.74 % on Poisson at $106\times$ the speed of CG at 4096². A natural next step is a coordinate-network bank for moving-frame flows, quadrature in three dimensions, and advection-dominated three-dimensional problems.

REPRODUCIBILITY STATEMENT

Every table and prose number is taken from hash-pinned run records identifying source, checkpoint, settings, cohort, allocation and audit (Appendix C); failed settings are kept.

AI USE STATEMENT

Language-model assistants supported method and experiment development, implementation, analysis, auditing and drafting. Recorded solver runs produced the numbers and the generators reproduce them. The authors are responsible for the methods, results and text.

REFERENCES

- J. Barnett and C. Farhat. Quadratic approximation manifold for mitigating the Kolmogorov barrier in nonlinear projection-based model order reduction. *Journal of Computational Physics*, 464:111348, 2022.
- P. Benner, S. Gugercin, and K. Willcox. A survey of projection-based model reduction methods for parametric dynamical systems. *SIAM Review*, 57(4):483–531, 2015.
- J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han. Once-for-All: Train one network and specialize it for efficient deployment. In *International Conference on Learning Representations*, 2020.
- K. Carlberg. Adaptive h -refinement for reduced-order models. *International Journal for Numerical Methods in Engineering*, 102(5):1192–1210, 2015.
- K. Carlberg, C. Bou-Mosleh, and C. Farhat. Efficient non-linear model reduction via a least-squares Petrov–Galerkin projection and compressive tensor approximations. *International Journal for Numerical Methods in Engineering*, 86(2):155–181, 2011.
- A. Cohen and R. DeVore. Approximation of high-dimensional parametric PDEs. *Acta Numerica*, 24:1–159, 2015.
- R. Geelen, S. Wright, and K. Willcox. Operator inference for non-intrusive model reduction with quadratic manifolds. *Computer Methods in Applied Mechanics and Engineering*, 403:115717, 2022.
- G. H. Golub and V. Pereyra. The differentiation of pseudo-inverses and nonlinear least squares problems whose variables separate. *SIAM Journal on Numerical Analysis*, 10(2):413–432, 1973.
- C. Greif and K. Urban. Decay of the Kolmogorov N -width for wave problems. *Applied Mathematics Letters*, 96:216–222, 2019.
- J. A. Hernández, M. A. Caicedo, and A. Ferrer. Dimensional hyper-reduction of nonlinear finite element models via empirical cubature. *Computer Methods in Applied Mechanics and Engineering*, 313:687–722, 2017.
- M. R. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436, 1952.
- Y. Kim, Y. Choi, D. Widemann, and T. Zohdi. A fast and accurate physics-informed neural network reduced order model with shallow masked autoencoder. *Journal of Computational Physics*, 451:110841, 2022.

- M. Kim, T. Wen, K. Lee, and Y. Choi. Physics-informed reduced order model with conditional neural fields. *arXiv preprint arXiv:2412.05233*, 2024.
- N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar. Neural operator: learning maps between function spaces. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- C. L. Lawson and R. J. Hanson. *Solving Least Squares Problems*. Prentice-Hall, Englewood Cliffs, NJ, 1974.
- K. Lee and K. T. Carlberg. Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. *Journal of Computational Physics*, 404:108973, 2020.
- Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.
- L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.
- D. W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963.
- B. Peherstorfer and K. Willcox. Online adaptive model reduction for nonlinear systems via low-rank updates. *SIAM Journal on Scientific Computing*, 37(4):A2123–A2150, 2015.
- O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.
- Y. Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, 2nd edition, 2003.
- L. Sirovich. Turbulence and the dynamics of coherent structures, Parts I–III. *Quarterly of Applied Mathematics*, 45(3):561–590, 1987.
- R. Ștefănescu and A. Sandu. Efficient approximation of sparse Jacobians for time-implicit reduced order models. *arXiv preprint arXiv:1402.2018*, 2014.
- M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, and M. Niepert. PDEBench: An extensive benchmark for scientific machine learning. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 35, 2022.
- M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in Neural Information Processing Systems*, 33, 2020.
- J. Weder, P. Schwerdtner, and B. Peherstorfer. Online-efficient model reduction on quadratic manifolds with Neural Galerkin schemes. *arXiv preprint arXiv:2412.17695*, 2024.
- H. Wu, H. Luo, H. Wang, J. Wang, and M. Long. Transolver: A fast transformer solver for PDEs on general geometries. In *International Conference on Machine Learning*, 2024.
- M. Yano and A. T. Patera. An LP empirical quadrature procedure for reduced basis treatment of parametrized nonlinear PDEs. *Computer Methods in Applied Mechanics and Engineering*, 344:1104–1123, 2019.
- J. Yu, L. Yang, N. Xu, J. Yang, and T. Huang. Slimmable neural networks. In *International Conference on Learning Representations*, 2019.

A METHOD DETAILS

This appendix gives the details of §3 that the main text leaves out: the ordering of the bank, the row scaling of each PDE, the Burgers stencil, the Navier–Stokes family, the quadrature fit and the solver constants. On the square the tests are eigenvectors of the Laplacian, $PK = \Lambda P$ with Λ diagonal, and every PDE scales the tested residual PR by the inverse of a diagonal matrix built from Λ . We write r_w for the scaled tested residual.

A.1 ORDERING THE BANK

Let $G = Q_G R_G$ be a thin QR factorisation. For every training field u_i we take its bank coefficients a_i and stack the scaled vectors $R_G a_i / \|u_i\|$ as the rows of a matrix $\mathcal{A}$. With the singular value decomposition $\mathcal{A} = USV^\top$ we set $T = R_G^{-1}V$ in (2). The columns of $\hat{G} = GT$ are then orthonormal and ordered by the singular values in S . Because T is invertible, the full-width bank gives the same trial states as before (to 10^{-10} in our runs), and the head is kept unchanged. The step uses training data only and has no trainable weights.

A.2 POISSON

The tested residual is scaled by Λ^{-1} . Because the retained tests are orthonormal and $Ku^* = F$, the scaled residual $\Lambda^{-1}P(K\tilde{u} - F) = P(\tilde{u} - u^*)$ is exactly the tested error, so the objective is the squared discrete L^2 error of the reduced state in the retained modes. In the span, the thin QR factorisation of $P\hat{G}_{R'}$ is computed once per mesh and width, and a query is one matrix–vector product and one triangular solve. At $R' = R$ the span solve reaches the bank floor in the tested norm.

A.3 HEAT

The tested Crank–Nicolson residual (7) is scaled by $(I + \frac{\Delta t \kappa}{2} \Lambda)^{-1}$, which turns each step into $P\tilde{u}_{n+1} = D P\tilde{u}_n$ with $D = \text{diag}((1 - \frac{\Delta t \kappa}{2} \lambda_i) / (1 + \frac{\Delta t \kappa}{2} \lambda_i))$: each tested mode is multiplied by its Crank–Nicolson factor. In the span the step is the linear recurrence

$$c_{n+1} = B_{R'}^+ D B_{R'} c_n, \quad (18)$$

with $B_{R'} = P\hat{G}_{R'}$ and c_0 fitted to the initial field. Because the step matrix $B_{R'}^+ D B_{R'}$ is fixed, its powers at the output times are formed once offline, so a query fits c_0 , applies one small matrix per output time and rebuilds the fields. This is exactly the Crank–Nicolson solution of the reduced problem at the full-order time step $\Delta t = 0.025$, and every heat result in the paper is computed this way.

A.4 BURGERS

The full-order problem uses the sign-upwind stencil

$$A(u)_{ij} = u_{ij} (\delta_x u + \delta_y u)_{ij}, \quad (\delta_x u)_{ij} = \frac{1}{\Delta x} \begin{cases} u_{ij} - u_{i-1,j}, & u_{ij} > 0, \\ u_{i+1,j} - u_{ij}, & u_{ij} \leq 0, \end{cases} \quad (19)$$

with δ_y analogous, zero ghost values on the walls and backward Euler in time. The tested residual (9) is scaled by $(I + \Delta t \nu \Lambda)^{-1}$, the modal form of the Helmholtz preconditioner the full-order Newton solver uses. Each step is warm-started from the previous coefficients, and a linear or quadratic extrapolation is used when its residual is smaller. At every mesh the step assembles the Jacobian analytically, clips the step, solves the damped normal equation by Cholesky factorisation and carries the damping between steps; the rows at 256^2 – 1024^2 also apply the test projections separably and, where faster, run as CUDA graphs, which reproduces the iterates to 6×10^{-12} with identical iteration counts.

A.5 NAVIER–STOKES

The unknown is the three-component, zero-mean velocity on the periodic unit cube. The initial velocity is the curl of two localised periodic vector potentials, each of the form $\exp[2 \sum_i (\cos(2\pi(x_i -$

$c_i)) - 1]$ along a fixed direction, with random common centre, separation in $[0.12, 0.24]$, relative strength in $[0.6, 1.4]$ and viscosity log-uniform in $[0.002, 0.01]$; the horizon is $T = 0.2$ with six output times. The data and the full-order comparator come from CNAB2 with two-thirds dealiasing. The bank is a rank-64 POD of training states, each centred on its own energy centroid; the centroid moves with the vortices, and a query solves for its three-component shift at every step together with the latent coordinates. All reduced tensors, including the quadratic term, are computed offline, so no quadrature is needed. The head is trained as an auto-decoder on the centred bank coefficients of training states. The span uses the first R' columns of this bank ordered by importance. The tests are the $M = 292$ lowest divergence-free Fourier modes. A query centres the initial field, projects it on the bank and, for the head, starts from a stored training code. It then takes 10 implicit-midpoint steps of $\Delta t = 0.02$, each solved by three damped Gauss–Newton sweeps with an analytic Jacobian, starting from a linear extrapolation of the last two steps. With the centring held fixed the same bank misses every case by far more than 5 %, so the centring is necessary.

A.6 EMPIRICAL QUADRATURE

On one state, a rule’s error is

$$\rho(u) = \frac{\|\sum_{i \in \mathcal{S}} w_i P_{:,i} A_i(u) - PA(u)\|_2}{\|PA(u)\|_2}, \quad \rho_{\max} \leq 0.116, \quad (20)$$

over held-out reachable states. The bar was fixed before any check was run. The fit (13) normalises each row, grows the support greedily by the Lawson–Hanson criterion with an exact non-negative refit after each addition, and stops at N_{eq} nodes. The fit states are states the dense solver reaches on fit trajectories; the held-out states come from trajectories disjoint from the fit and the evaluation cases. A deployed rule is its node indices, weights and the cached $N_{\text{eq}} \times 5 \times R$ bank block. The Burgers rows of Table 1 use a rule that needs no fit: the uniform interior lattice with 64 intervals per axis, $63 \times 63 = 3969$ nodes with equal weights, which are the same physical points on every mesh. It is checked on reached states in the same way as a fitted rule.

A.7 SOLVER CONSTANTS

Each Levenberg–Marquardt attempt solves $(H + \lambda \text{diag}(\text{diag } H)) \delta = -g$ with $H = J^\top J$ and $g = J^\top r_w$, and accepts δ only if the residual decreases. The stopping test is scale free, $\|J^\top r_w\|_2 \leq \eta_{\text{tol}} \|J\|_F \|r_w\|_2$, so one tolerance serves every mesh and width. Damping starts at $\lambda_0 = 10^{-6}$ (Poisson, Burgers) or 10^{-4} (heat), is divided by 3 on acceptance and multiplied by 10 on rejection. Head solves start from a stored training code. A solve that stalls is reported as early-stopped, separately from one that meets the stopping test. The *bank floor* is the projection error onto the span of the bank, which no solve can beat.

B NEURAL OPERATORS

We trained an FNO (Li et al., 2021), a U-Net (Ronneberger et al., 2015), a DeepONet (Lu et al., 2021) and a Transolver (Wu et al., 2024) on Burgers 2D, heat 2D and Navier–Stokes 3D, each at the mesh it is evaluated on, with AdamW, one fixed seed per network (Table B.2) and a 3000 s wall budget per network, and scored them by the same metric as our model. They produce the requested output times in one pass, so their output times are fixed at training. An operator’s own evaluation resolution can also trade accuracy for cost (§2); here each trained operator has one evaluation setting.

Other reduced-order models at 1024^2 . Table B.1 gives POD-LSPG and a quadratic manifold (Geelen et al., 2022) at several sizes, from the same job as Table 2. Neither is both more accurate and faster than our setting. POD-LSPG needs 256 or more unknowns to approach 1 %; the quadratic manifold stays above 7 % up to 64.

Navier–Stokes cells. Both sides draw from the same 512 training trajectories. The operators train on 448 and choose their size on the other 64, the same split the head used. The comparison is on the 16 development cases of Table 1. Under the 3000 s budget the operators trained for 269–624 epochs at 32^3 , 80–232 at 64^3 and 62–510 at 96^3 (Table B.2). At 32^3 CNAB2 with 40 steps reaches 0.152 %,

Table B.1: Burgers 2D at 1024^2 , one allocation (job 4204019, A100 80 GB, six development cases): worst and median evolved same-grid error, median GPU query time, and speedup over the one full-order setting used for this cell in Tables 2 and 3 (Newton–BiCGStab, tol 3×10^{-3} , $\Delta t = 0.005$, 70.5 ms, the fastest setting at least as accurate as the NM-ROM accurate setting); below $1 \times$ the row is slower. FOM rows are Newton–BiCGStab settings.

Method	Unknowns	Worst (%)	Median (%)	GPU ms	Speedup
NM-ROM, span ($R' = 384$)	384	0.21	0.065	318	$0.22 \times$
NM-ROM, head ($k = 16$)	16	2.29	1.04	34.9	$2.02 \times$
POD-LSPG	16	29.3	22.1	178	$0.40 \times$
POD-LSPG	64	7.41	3.66	911	$0.077 \times$
POD-LSPG	256	1.10	0.42	12147	$0.006 \times$
POD-LSPG	512	0.44	0.053	47503	$0.001 \times$
Quadratic manifold	16	22.6	15.0	733	$0.096 \times$
Quadratic manifold	32	13.0	6.68	2547	$0.028 \times$
Quadratic manifold	64	7.26	3.58	8257	$0.009 \times$
FOM, tol 10^{-2} , $\Delta t = 0.01$	1023^2	3.46	1.47	34.0	—
FOM, tol 10^{-2} (lean), $\Delta t = 0.01$	1023^2	1.66	1.20	37.0	—
FOM, tol 3×10^{-3} (lean), $\Delta t = 0.005$	1023^2	0.048	0.038	70.5	—

just above the accurate setting’s 0.151 %, so the rule selects 50 steps; against 40 steps the accurate setting would run at $0.95 \times$. At 96^3 the comparison is on 32 held-out cases.

How much data each side was given. On Burgers the operators were trained on 128 trajectories and our model on 4 608. The gap comes from the cost of one sample, not from a choice. Both sides draw initial conditions from the same family and are scored at the same output times, but at 256^2 each operator target is a 4096-interval solution restricted to the 256-interval grid, which costs about 670 times more than a trajectory behind our bank and head. At 1024^2 and 2048^2 the same 128 trajectories are recomputed on the evaluation mesh with a tight Newton tolerance, so there the operators are trained on exactly the fields they are scored against, while our model, trained at 256^2 , is used unchanged. Matching 4 608 trajectories at that fidelity would cost about 163 A100-hours. An operator also sees one input–output pair per trajectory (640 supervised states), while the head is fitted per state (131 072 states).

C EXPERIMENTAL CONFIGURATION AND REPRODUCIBILITY

Tables C.2 and C.3 identify the two-dimensional studies and Table C.7 the three-dimensional ones. Within each study, training precedes evaluation, and the selected bank, its ordering and the head remain frozen. Deployment changes reduced coordinates, not network weights. Table C.1 gives the training configuration of every frozen model.

Full-order and solver details. The Burgers FOM is Newton’s method with BiCGStab inner solves (at most 200 iterations, Jacobian–vector products by forward-mode differentiation) preconditioned by the exact inverse of the implicit Helmholtz operator $(I + \Delta t \nu \lambda)^{-1}$ (advection dropped); Newton stops when the residual norm falls below the nonlinear tolerance times the previous state’s norm, or after 20 iterations, and always takes the full step (no line search or damping). Each named setting fixes Δt , the nonlinear and the linear tolerance.

C.1 HEADLINE SETTINGS, TIMES AND TIMING PROTOCOL

The settings behind Table 1, all from one frozen model per problem with no retraining: Poisson (square and cube) and heat use the bank span at two widths ($R' = 512/128$ square, $128/64$ cube, $128/48$ heat 2D, $320/128$ heat 3D, Crank–Nicolson stepping); Navier–Stokes uses the head $k = 8$ and the span $R' = 16$ (the cheapest span setting below 5 %); the L-shape uses the span $R' = 128/64$ (the widest span its tests support is 128); Burgers uses the span $R' = 384$ and $R' = 128$ at every mesh. Burgers 3D uses one coordinate bank ($R = 512$, trained once on 32^3 , 64^3 and 128^3 fields) with the span $R' = 512$ and $R' = 192$ ($R' = 256$ at 256^3), a fixed-sweep Levenberg–Marquardt

Table B.2: Training of the neural operators of Table 2, one row per trained network shown there: parameters, epochs completed within the 3000 s wall budget, and why training stopped. Every network used AdamW, weight decay 10^{-4} , batch 8 (by gradient accumulation where memory required it), seed 20260914 and the checkpoint of its best validation epoch. Training sets: Burgers 128 trajectories (32 validation), heat 512 (16 validation), Navier–Stokes 448 (64 validation). The Burgers 2048² FNO was stopped by host memory after 19 epochs.

Cell	Operator	Parameters	Epochs	Stopped by
Burgers 2D, 256 ²	FNO	17,877,317	692	wall budget
	U-Net	17,462,021	908	wall budget
	Transolver	3,108,240	1628	wall budget
	DeepONet	2,569,349	537	early stopping
Burgers 2D, 1024 ²	FNO	17,877,317	37	wall budget
	U-Net	7,763,461	223	wall budget
	Transolver	3,108,240	131	wall budget
	DeepONet	2,569,349	308	wall budget
Burgers 2D, 2048 ²	FNO	17,877,317	19	host memory
	U-Net	7,763,461	62	wall budget
	Transolver	3,108,240	24	wall budget
	DeepONet	2,569,349	69	wall budget
Heat 2D, 1024 ²	FNO	17,877,189	7	wall budget
	U-Net	7,763,173	71	wall budget
	Transolver	12,746,048	228	wall budget
	DeepONet	2,568,917	98	wall budget
Heat 2D, 2048 ²	FNO	17,877,189	6	wall budget
	U-Net	7,763,173	23	wall budget
	Transolver	14,126,144	394	wall budget
	DeepONet	2,568,917	31	wall budget
Navier–Stokes 3D, 32 ³	FNO	18,888,495	624	wall budget
	U-Net	22,581,327	458	wall budget
	Transolver	6,003,816	269	wall budget
	DeepONet	5,741,519	298	early stopping
Navier–Stokes 3D, 64 ³	FNO	18,888,495	88	wall budget
	U-Net	22,581,327	80	wall budget
	Transolver	17,996,928	110	wall budget
	DeepONet	5,741,519	232	wall budget
Navier–Stokes 3D, 96 ³	FNO	18,888,495	110	wall budget
	U-Net	22,581,327	62	wall budget
	Transolver	6,619,288	510	wall budget
	DeepONet	5,741,519	70	wall budget

Table C.1: Training configurations of the frozen checkpoints, transcribed from the committed configuration and code files of each study (paths and SHA256 in evidence/training-configs-2026-09-21).

Model	Stage	Network	Optimiser, learning rate	Steps	Batch	Training data
Burgers 2D ($k = 16$, $R = 512$)	bank	128 Fourier features (scale 4), width 1024, 2 layers	AdamW, wd 10^{-5} , warm-up + cosine $10^{-3} \rightarrow 10^{-5}$	300000	all states $\times$ 4096 points	16384 states of 576 trajectories
	head	width 512, 2 layers	Adam, warm-up + cosine $10^{-3} \rightarrow 10^{-5}$	200000	4096 states	131072 states of 4608 trajectories
Poisson 2D ($k = 32$, $R = 512$)	bank	64 Fourier features (scale 4), width R , 2 layers	Adam, four phases at 10^{-3} , 3×10^{-4} , 3×10^{-4} , 10^{-4}	100000, 40000, 30000, 30000	64 sources $\times$ 4096 points	2611 fit / 461 validation sources as bank
	head	width 128, 2 layers	Adam, 10^{-3} as Poisson $k = 32$	150000 (full batch)	all	2611 fit / 461 validation sources
L-shape ($k = 16$, $R = 512$)	bank / head	64 Fourier features (scale 4), 2 layers; head width 128, 2 layers	Adam, cosine (floor 0.03); 10^{-3} as Poisson $k = 32$	100000, 40000, 30000, 30000; head 150000	64 sources $\times$ 4096 points	2611 fit / 461 validation sources
Poisson 3D ($k = 16$, $R = 128$)	bank / head	64 Fourier features (scale 1.5), widths 256; head width 256 + skip	Adam, cosine (floor 0.03); 10^{-3} bank, 5×10^{-4} head	150000 / 100000	64 states $\times$ 2048 points / 128 states	512 training / 16 validation fields
Heat 2D ($k = 8$, $R = 128$)	bank / head	32 Fourier features (scale 1.5), width 256; head width 256	Adam, clip 1, exact coefficient re-fit every 10000 steps; 10^{-3} bank, 5×10^{-4} head	80000 / 80000	64 states $\times$ 2048 points / 128 states	512 training / 16 validation draws at 64 ²
Heat 3D ($k = 32$, $R = 320$)	bank / head	128 Fourier features (scale 2), width 512, 3 layers; head width 512, 2 layers + skip	bank: Adam, clip 1, warm-up + cosine $10^{-3} \rightarrow 10^{-5}$; head: Adam, cosine from 5×10^{-4} (floor 0.03)	40000 (step kept) / 120000	512 states on the full 32 ³ grid / 256 states	2048 training / 256 validation trajectories (6 times each)
Burgers 3D ($R = 512$, span only)	bank	128 Fourier features (scale 2), width 1024, 3 layers	Adam, clip 1, warm-up + cosine $10^{-3} \rightarrow 10^{-5}$	12000 (step 7000 kept)	128 states $\times$ 32768 points per mesh	18432 states of 1536 trajectories at each of 32 ³ , 64 ³ , 128 ³ ; 96 validation trajectories
Navier–Stokes 3D ($k = 8$, $R = 64$)	bank	POD of centred snapshots, no trained weights	—	—	—	768 states of 128 trajectories
	head	width 256, 2 layers + skip	Adam, clip 1, warm-up + cosine $10^{-3} \rightarrow 10^{-5}$	100000 (full batch)	all	9408 training / 1344 validation states of 512 trajectories

Table C.2: Problem specification. Cohort and reduced sizes are read from the run configurations where recorded; rows describe their recorded development, validation and held-out cohorts.

PDE	equation, domain, boundary	meshes	time stepping	reduced sizes	reference	cohorts
Burgers 2D	$u_t + u(u_x + u_y) = \nu \Delta u$, $(0, 1)^2$, $u _{\partial\Omega} = 0$	256^2 – 4096^2	$\Delta t = 0.005$, backward Euler, sign-upwind	$k = 16$, $R = 512$	refined 4096 ² , $\Delta t = 0.00015625$	6 development cases; 32 validation; 64 held-out at 256^2 , 512^2 , 1024^2 , 4096^2
Poisson 2D	$-\Delta u = f$, $(0, 1)^2$, $u _{\partial\Omega} = 0$	256^2 – 4096^2	none (elliptic)	$k = 32$, $R = 512$	exact discrete solution; 2048 ² refinement	12 development sources
Heat 2D, wide bank	$u_t = \kappa \Delta u$, $(0, 1)^2$	1024^2 – 4096^2	Crank–Nicolson, $\Delta t = 0.025$	$k = 8$, $R = 128$	exact same-grid propagation	16 validation cases (selection); 16 held-out cases
Poisson, L-shape	$-\Delta u = f$, $(0, 1)^2 \setminus [\frac{1}{2}, 1)^2$	256^2 – 2048^2	none	$k = 16$, $R = 512$	converged discrete solution	3072 / 256 / 32 sources (train / selection / development)

Table C.3: Sampled problem families.

PDE	equation and boundary	sampled family
Burgers 2D	$u_t + u(u_x + u_y) = \nu \Delta u$ on $(0, 1)^2$, $u = 0$ on $\partial\Omega$	$u_0 = a \exp(- x - c ^2/2w^2)$, $c_i \sim U(0.15, 0.85)$, $w \sim U(0.05, 0.20)$, $a \sim U(0.5, 2)$; $\nu \sim \log U(0.01, 0.1)$; outputs $t \in \{0.05, \dots, 0.25\}$
Burgers 3D	$u_t + u(u_x + u_y + u_z) = \nu \Delta u$ on $(0, 1)^3$, $u = 0$ on $\partial\Omega$	$u_0 \propto a \mu(x) \sum_{b \leq B} \rho_b \exp(- x - c_b ^2/2w_b^2)$, $\mu = 64 \prod_i x_i(1 - x_i)$, $B \in \{1, 2, 3\}$, $c_{b,i} \sim U(0.2, 0.8)$, $w_b \sim U(0.10, 0.20)$, $\rho_b \sim U(0.5, 1)$, peak normalised to $a \sim U(0.5, 2)$; $\nu \sim \log U(0.01, 0.1)$; backward Euler, sign-upwind; outputs $t \in \{0.05, \dots, 0.25\}$
Poisson 2D	$-\Delta u = f$ on $(0, 1)^2$, $u = 0$ on $\partial\Omega$	$f = a \exp(- x - c ^2/2w^2)$, $c_i \sim U(0.15, 0.85)$, $w \sim \log U(0.02, 0.1)$, $a \sim U(0.5, 2)$
Poisson, L-shape	same source family on $(0, 1)^2 \setminus [\frac{1}{2}, 1)^2$	as above; sources centred in the removed quadrant rejected
Heat 2D	$u_t = \kappa \Delta u$ on $(0, 1)^2$, $u = 0$ on $\partial\Omega$, $\kappa = 0.02$ fixed	$u_0 = a 16x_1(1 - x_1)x_2(1 - x_2) \exp(- x - c ^2/2w^2)$, $c_i \sim U(0.35, 0.65)$, $w \sim U(0.10, 0.15)$, $a \sim U(0.8, 1.2)$; outputs $t \in \{0, 0.1, \dots, 0.5\}$; Crank–Nicolson $\Delta t = 0.025$

solve and a precomputed quadratic advection tensor (about $4R'$ tests); its settings were chosen by the same rule on 64 validation cases and its rows are 32 sealed held-out cases, all on H200 GPUs. At 128^3 the FOM setting chosen on validation is less accurate than the accurate setting on the held-out cases, so the rule picks a tighter Newton tolerance; against the validation choice (27.4 ms) the fast setting is $2.87\times$ faster. The Burgers 4096² row was timed on an H200 GPU and the other Burgers rows on A100 80 GB GPUs; the reduced query barely depends on the mesh, so the faster GPU puts 4096² below 2048². Over evolved times the accurate three-dimensional heat error is 0.083 %. At 128^3 and 256^3 the three-dimensional Poisson rows use 16 development cases. The FOM setting can differ between meshes of a series.

Table C.4 gives the settings and measured times behind every row of Table 1. Every cost and error come from the same invocation. Timings use GPU warm-up and synchronisation, double precision and the highest matrix-multiplication precision; all repetitions, including outliers, enter the median, and no time is borrowed from another allocation. Requested full fields and initialisation are charged; host transfer is included only where the table says complete query.

The Poisson and three-dimensional heat CG comparators are unpreconditioned; heat CG is warm-started from the previous time level.

Three-dimensional Poisson ($-\Delta u = f$, Gaussian sources) and heat ($u_t = \kappa \Delta u$, Gaussian initial fields) are posed on the unit cube with zero Dirichlet walls and extend the operators of Appendix A across three axes. Navier–Stokes is described in Appendix A.5; its FOM is the pseudo-spectral CNAB2 solver, not a CG solve.

Training/evaluation separation (Burgers). The 4608 training trajectories of the Burgers checkpoint were regenerated from their seeds and compared with every evaluation cohort (the 32 validation cases of Table 3, the 64 held-out cases and the 6 development cases): no cohort shares a parameter vector, or any single parameter value, with the training set.

D TUNABILITY LADDERS

Table D.1 gives the bank-width ladders behind §6.3.

Table C.4: Supporting data for Table 1, including the full-order setting the rule selected for each row: the two settings of each frozen model, median times (ms), timing scope and status. “EQ” is the lattice rule of Appendix A.6. Where a run also recorded the complete query (host transfer included), those speedups are listed. Heat errors are measured against the exact time evolution on the same grid, so the NM-ROM and CN-CG errors both include the Crank–Nicolson time error.

Problem	Mesh	Accurate	Fast	Accurate ms	Fast ms	FOM setting	FOM ms	Timing	Complete query: acc. / fast	Status
Poisson 2D	256 ²	span, $R'=512$	span, $R'=128$	0.77	0.59	CG, mol 3x10 ⁻²	10.28	GPU query	7.05x / 7.72x (complete query)	development (timing provisional)
Poisson 2D	1024 ²	span, $R'=512$	span, $R'=128$	3.54	1.44	CG, mol 3x10 ⁻²	98.75	GPU query	14.9x / 21.3x (complete query)	development (timing provisional)
Poisson 2D	2048 ²	span, $R'=512$	span, $R'=128$	12.01	3.74	CG, mol 10 ⁻¹	650.24	GPU query	26.0x / 38.5x (complete query)	development
Poisson 2D	4096 ²	span, $R'=512$	span, $R'=128$	42.39	12.54	CG, mol 10 ⁻¹	4490.02	GPU query	46.3x / 66.2x (complete query)	development
Poisson, L-shape 2D	256 ²	span, $R'=128$	span, $R'=64$	0.32	0.29	CG, mol 3x10 ⁻²	15.25	GPU query	—	development
Poisson, L-shape 2D	512 ²	span, $R'=128$	span, $R'=64$	0.61	0.53	CG, mol 3x10 ⁻²	24.23	GPU query	—	development
Poisson, L-shape 2D	1024 ²	span, $R'=128$	span, $R'=64$	1.82	1.54	CG, mol 3x10 ⁻²	89.14	GPU query	—	development
Poisson, L-shape 2D	2048 ²	span, $R'=128$	span, $R'=64$	3.69	2.99	CG, mol 3x10 ⁻²	311.12	GPU query	—	development
Heat 2D	1024 ²	span, $R'=128$	span, $R'=48$	0.64	0.39	CN-CG, $\Delta t=0.025$, mol 10 ⁻⁴	67.00	GPU query	—	held-out
Heat 2D	2048 ²	span, $R'=128$	span, $R'=48$	1.96	1.05	CN-CG, $\Delta t=0.025$, mol 10 ⁻⁴	424.37	GPU query	—	held-out
Heat 2D	4096 ²	span, $R'=128$	span, $R'=48$	8.42	4.74	CN-CG, $\Delta t=0.025$, mol 10 ⁻⁴	1945.97	GPU query	—	held-out (timing provisional)
Burgers 2D	256 ²	span, $R'=384$, $M=1536$, EQ	span, $R'=128$, $M=512$, EQ	45.71	17.44	Newton-BiCGStab, tol 3x10 ⁻³	16.87	GPU query	—	development
Burgers 2D	512 ²	span, $R'=384$, $M=1536$, EQ	span, $R'=128$, $M=512$, EQ	45.91	17.39	Newton-BiCGStab, tol 3x10 ⁻³	27.70	GPU query	—	development
Burgers 2D	1024 ²	span, $R'=384$, $M=1536$, EQ	span, $R'=128$, $M=512$, EQ	49.52	21.31	Newton-BiCGStab, tol 3x10 ⁻³	72.14	GPU query	—	development
Burgers 2D	2048 ²	span, $R'=384$, $M=1536$, EQ	span, $R'=128$, $M=512$, EQ	90.09	35.15	Newton-BiCGStab, tol 3x10 ⁻³	297.30	GPU query	—	development
Burgers 2D	4096 ²	span, $R'=384$, $M=1536$, EQ	span, $R'=128$, $M=512$, EQ	59.60	24.06	Newton-BiCGStab, tol 3x10 ⁻³	523.13	GPU query	—	development
Poisson 3D	32 ³	span, $R'=128$	span, $R'=64$	0.32	0.29	CG, mol 10 ⁻²	1.41	GPU query	—	held-out
Poisson 3D	64 ³	span, $R'=128$	span, $R'=64$	0.44	0.35	CG, mol 10 ⁻²	3.57	GPU query	2.78x / 3.01x (complete query)	held-out
Poisson 3D	128 ³	span, $R'=128$	span, $R'=64$	0.91	0.58	CG, mol 10 ⁻²	12.42	GPU query	3.63x / 3.90x (complete query)	development
Poisson 3D	256 ³	span, $R'=128$	span, $R'=64$	6.13	3.50	CG, mol 10 ⁻²	142.20	GPU query	3.32x / 3.52x (complete query)	development
Heat 3D	32 ³	span, $R'=320$	span, $R'=128$	0.31	0.22	CN-CG, $\Delta t=0.025$, mol 10 ⁻⁴	3.38	GPU query	—	held-out (timing provisional)
Heat 3D	64 ³	span, $R'=320$	span, $R'=128$	0.64	0.38	CN-CG, $\Delta t=0.025$, mol 10 ⁻⁴	5.42	GPU query	—	held-out (timing provisional)
Heat 3D	128 ³	span, $R'=320$	span, $R'=128$	2.99	1.37	CN-CG, $\Delta t=0.025$, mol 10 ⁻⁴	17.33	GPU query	—	held-out (timing provisional)
Heat 3D	256 ³	span, $R'=320$	span, $R'=128$	23.61	10.68	CN-CG, $\Delta t=0.025$, mol 10 ⁻⁴	179.42	GPU query	—	held-out
Burgers 3D	64 ³	span, $R'=512$, $\Delta t=0.005$, tensor	span, $R'=192$, $\Delta t=0.01$, tensor	81.11	8.28	Newton-BiCGStab, $\Delta t=0.005$, tol 10 ⁻³	17.26	GPU query	—	sealed held-out
Burgers 3D	128 ³	span, $R'=512$, $\Delta t=0.005$, tensor	span, $R'=192$, $\Delta t=0.01$, tensor	85.00	9.54	Newton-BiCGStab, $\Delta t=0.01$, tol 10 ⁻³	48.68	GPU query	—	sealed held-out
Burgers 3D	256 ³	span, $R'=512$, $\Delta t=0.01$, tensor	span, $R'=256$, $\Delta t=0.01$, tensor	90.70	35.50	Newton-BiCGStab, $\Delta t=0.01$, tol 10 ⁻²	198.69	GPU query	—	sealed held-out
Navier–Stokes 3D	32 ³	head, $k=8$	bank span, $R'=16$	6.26	3.65	CNAB2 (spectral), 50 steps	6.84	GPU query	—	development
Navier–Stokes 3D	64 ³	head, $k=8$	bank span, $R'=16$	8.19	4.64	CNAB2 (spectral), 50 steps	23.65	GPU query	—	development
Navier–Stokes 3D	96 ³	head, $k=8$	bank span, $R'=16$	15.96	9.16	CNAB2 (spectral), 60 steps	106.43	GPU query	—	development

Table C.5: Solver iterations per query and where the query time goes. On Poisson and heat the query is a direct solve; *input* is projecting the forcing or fitting the initial state, *solve* the reduced solve including all time steps, and *output* rebuilding the field at the output times. These stage times come from separately compiled stages in the same jobs, so they do not add exactly to the query time. Burgers: Levenberg–Marquardt iterations per query, median over the cases (largest number in one step in brackets): in two dimensions over 50 time steps of the six development cases, every step stopped on the convergence test; in three dimensions over 50 ($\Delta t = 0.005$) or 25 ($\Delta t = 0.01$) steps of the 32 held-out cases. Navier–Stokes: 10 time steps of three Gauss–Newton sweeps each, plus 12 Gauss–Newton sweeps that fit the head’s starting code to the projected initial field, starting from a stored training code. Query times as in Table C.4 (ms).

Problem	Mesh	Setting	Iterations	Input	Solve	Output	Query
Poisson 2D	4096 ²	span $R'=512$	0	0.99	<0.01	41.79	42.39
		span $R'=128$	0	0.74	<0.01	12.52	12.54
Heat 2D	4096 ²	span $R'=128$	0	1.34	0.07	6.74	8.42
		span $R'=48$	0	1.10	0.07	3.09	4.74
Heat 3D	256 ³	span $R'=320$	0	11.88	0.08	12.28	23.61
		span $R'=128$	0	5.81	0.08	5.42	10.68
Burgers 2D	256 ²	span $R'=384$	52 (4)				45.71
		span $R'=128$	38.5 (4)				17.44
	512 ²	span $R'=384$	52 (4)				45.91
		span $R'=128$	38.5 (4)				17.39
	1024 ²	span $R'=384$	54 (5)				49.52
		span $R'=128$	51.5 (5)				21.31
	2048 ²	span $R'=384$	54 (5)				90.09
		span $R'=128$	51.5 (5)				35.15
	4096 ²	span $R'=384$	54 (5)				59.60
		span $R'=128$	51.5 (5)				24.06
Burgers 3D	64 ³	span $R'=512$	51 (3)				81.11
		span $R'=192$	27 (5)				8.28
	128 ³	span $R'=512$	51 (3)				85.00
		span $R'=192$	26 (5)				9.54
	256 ³	span $R'=512$	28 (5)				90.70
		span $R'=256$	27 (5)				35.50
Navier–Stokes 3D	32 ³	head $k=8$	30 + 12				6.26
		span $R'=16$	30				3.65
	64 ³	head $k=8$	30 + 12				8.19
		span $R'=16$	30				4.64
Navier–Stokes 3D	96 ³	head $k=8$	30 + 12				15.96
		span $R'=16$	30				9.16

Table C.6: Supporting data for Table 3: every row of that table with its residual path and median GPU query time from the same allocation. Our rows use the unoptimised dense reference path (no empirical quadrature, stopping tolerance 10^{-6}); our optimised query is timed only in Table 1, in other jobs, and no ratio across jobs is formed. The shallow masked-autoencoder NM-LSPG of Kim et al. (2022) passed its reproduction gate, on their own 2D Burgers benchmark (their Sec. 6.2), on the third of three pre-registered attempts (sigmoid activation; median 1.45 % against a 1.5 % bar; published <1 %). Its published encoder width ($2n$) needs 136 GB to train at 256^2 and 2183 GB at 512^2 , beyond the 77 and 135 GB devices; at 512^2 its encoder, capped at width 4096, trained only 229–231 epochs in its 4800 s budget, so those errors partly reflect a training-time limit. The data-matched row is trained on our bank’s 576 trajectories.

Mesh	Method	Residual path	k	Worst (%)	Median (%)	GPU ms (same job)	MB
256^2	Kim et al. NM-LSPG	dense	8	163.93	43.34	1301	2480
256^2	Kim et al. NM-LSPG	dense	16	145.40	43.87	2384	2567
256^2	Kim et al. NM-LSPG	dense	32	127.51	43.61	3846	2742
256^2	Kim et al. NM-LSPG, data-matched	dense	16	120.33	38.71	3886	2567
256^2	POD-LSPG	dense	8	55.52	22.72	138	18
256^2	POD-LSPG	dense	16	41.53	10.45	259	30
256^2	POD-LSPG	dense	32	24.78	5.65	519	55
256^2	This work, head ($k = 16$)	dense (reference path)	16	6.79	0.66	262	625
512^2	Kim et al. NM-LSPG	dense	8	175.71	55.48	746	9959
512^2	Kim et al. NM-LSPG	dense	16	174.99	58.56	1497	10511
512^2	Kim et al. NM-LSPG	dense	32	252.33	56.72	2059	10879
512^2	POD-LSPG	dense	8	55.82	24.08	314	71
512^2	POD-LSPG	dense	16	42.01	13.83	617	122
512^2	POD-LSPG	dense	32	25.63	7.74	383	222
512^2	This work, head ($k = 16$)	dense (reference path)	16	7.72	0.69	404	2368

Table C.7: Three-dimensional configurations, read from the run records. Domains are the unit cube with homogeneous Dirichlet data, except Navier–Stokes, which is periodic.

Problem	Mesh	Cases	Reduced model	Named FOM	Residual
Poisson 3D	32^3 – 256^3	64 held-out (32^3 , 64^3); 16 development (128^3 , 256^3)	span $R' \in \{16, \dots, 128\}$	CG, rtol 10^{-2} , no preconditioner	exact (linear)
Heat 3D	32^3 – 256^3	64 held-out (all times)	span $R' \in \{320, \dots, 32\}$ ($R = 320$)	CN-CG, $\Delta t = 0.025$, rtol 10^{-4} (the rule’s choice at every mesh)	exact (linear)
Burgers 3D	64^3 , 128^3 , 256^3	64 validation (settings); 32 scaled held-out	coordinate bank $R = 512$; span $R' \in \{512, \dots, 64\}$, $\Delta t \in$ $\{0.005, 0.01\}$	Newton-BICGSTab, setting per row by the rule	precomputed quadratic tensor, checked on reached states
Navier–Stokes 3D	32^3 , 64^3 , 96^3 periodic	16 development + 32 held-out cases (96^3)	POD bank $R = 64$ in a moving frame; head $k = 8$; span $R' \in$ $\{64, 48, 32, 16, 8\}$	CNAB2 (spectral), 50–70 steps (per row by the rule)	exact (precomputed quadratic tensor)

Table D.1: Bank-width tunability at the finest mesh. Each block is one frozen model, and within a cohort every speedup divides the median time of *one* full-order setting (the *FOM* row, the fastest at least as accurate as the block’s most accurate setting, in the same allocation) by the setting’s time (ms). Error is the worst same-grid relative L^2 error (%), over evolved times for Navier–Stokes and Burgers and over all times for heat. Held-out cases were never used for selection and share the development settings; for Burgers 3D the development column is the 64 validation cases on which its settings were chosen, and only the two selected widths were run on the held-out cases. The Burgers 2D held-out cases ran on an A100 GPU and the development cases on an H200 GPU. Heat is shown at 2048^2 , the finest mesh whose timing is not provisional.

Setting	Development cohort			Held-out cohort		
	Err. (%)	ms	Speedup	Err. (%)	ms	Speedup
<i>Poisson 2D, ordered bank, 4096^2 (12 development cases)</i>						
Head, $k=32$, $R'=512$	3.15	43.4	103×	—	—	—
Bank span, $R'=512$	0.74	42.4	106×	—	—	—
Bank span, $R'=384$	0.77	32.5	138×	—	—	—
Bank span, $R'=256$	0.94	22.6	199×	—	—	—
Bank span, $R'=128$	2.31	12.5	358×	—	—	—
Bank span, $R'=64$	5.33	7.1	631×	—	—	—
Bank span, $R'=32$	9.17	4.4	1022×	—	—	—
<i>FOM</i> : CG, $\text{rtol } 10^{-1}$	0.45	4490	1×	—	—	—
<i>Navier–Stokes 3D, co-moving frame, ordered bank, 96^3 (16 development / 32 held-out cases)</i>						
Head, $k=8$	0.15	16.0	6.67×	0.21	15.8	7.79×
Bank span, $R'=64$	0.62	15.4	6.92×	1.12	15.3	8.05×
Bank span, $R'=48$	1.00	12.9	8.27×	1.26	12.6	9.76×
Bank span, $R'=32$	1.26	11.6	9.14×	1.46	11.5	10.7×
Bank span, $R'=16$	2.96	9.2	11.6×	3.24	9.2	13.4×
Bank span, $R'=8$	5.55	8.2	13.0×	6.91	8.1	15.2×
<i>FOM</i> : CNAB2 (spectral), 60 / 70 steps	0.063	106	1×	0.056	123	1×
<i>Burgers 2D, ordered bank, 4096^2 (6 development / 64 held-out cases)</i>						
Head, $k=16$, $R'=512$	2.41	41.5	12.6×	—	—	—
Bank span, $R'=512$ (rule not confirmed)	0.30	77.6	6.74×	—	—	—
Bank span, $R'=384$	0.22	59.6	8.78×	1.35	121	10.6×
Bank span, $R'=256$	0.74	42.2	12.4×	—	—	—
Bank span, $R'=128$	1.89	24.1	21.7×	3.11	46.7	27.3×
Bank span, $R'=64$	4.83	16.1	32.6×	—	—	—
Bank span, $R'=32$	10.7	12.1	43.2×	—	—	—
<i>FOM</i> : Newton–BiCGStab, $\text{tol. } 3 \times 10^{-3}$	0.050	523	1×	0.14	1275	1×
<i>Burgers 3D, ordered bank, $\Delta t=0.01$, 256^3 (64 validation / 32 held-out cases)</i>						
Bank span, $R'=512$	2.43	89.7	2.22×	2.20	90.7	2.19×
Bank span, $R'=384$	3.10	58.4	3.41×	—	—	—
Bank span, $R'=256$	3.84	34.0	5.86×	3.77	35.5	5.60×
Bank span, $R'=192$	5.65	25.7	7.74×	—	—	—
Bank span, $R'=128$	10.5	18.0	11.1×	—	—	—
Bank span, $R'=96$	14.4	13.9	14.3×	—	—	—
Bank span, $R'=64$	21.9	9.5	20.9×	—	—	—
<i>FOM</i> : Newton–BiCGStab, $\Delta t=0.01$, $\text{tol. } 10^{-2}$	1.27	199	1×	1.52	199	1×
<i>Heat 2D, ordered bank, Crank–Nicolson, $\Delta t=0.025$, 2048^2 (16 held-out cases)</i>						
Head, $k=8$, $R'=128$	—	—	—	1.36	31.5	13.5×
Bank span, $R'=128$	—	—	—	0.13	1.96	216×
Bank span, $R'=96$	—	—	—	0.15	1.63	261×
Bank span, $R'=64$	—	—	—	0.19	1.28	332×
Bank span, $R'=48$	—	—	—	0.28	1.05	405×
Bank span, $R'=32$	—	—	—	1.08	0.83	512×
Bank span, $R'=16$	—	—	—	8.68	0.62	687×
<i>FOM</i> : CN–CG, $\Delta t = 0.025$, $\text{rtol } 10^{-4}$	—	—	—	0.045	424	1×